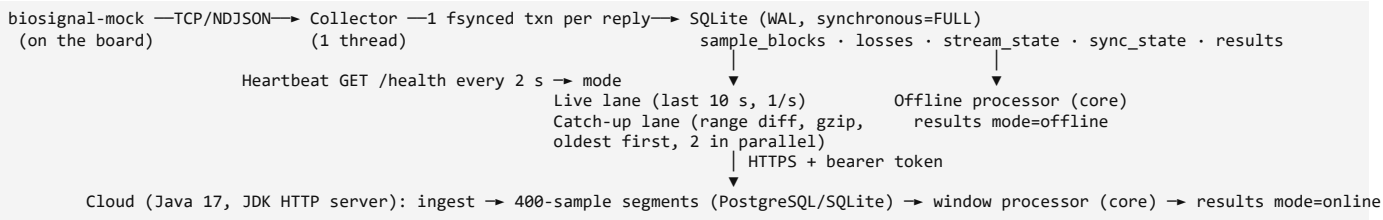
Design note

Validated against stand-in versions of `biosignal-mock` and `check_ground_truth` written from the assessment's "Mock service" section; the official tools were not available. **A real Raspberry Pi 4 run was not possible:** ARM64 figures come from a GitHub Actions ubuntu-24.04-arm native ARM64 runner, limited to 4 CPUs / 2 GB; not a Raspberry Pi 4; power cuts simulated by SIGKILL.

Architecture



Device. The collector polls each sensor every 0.5 s (back to back while behind), always asking `from_index = next expected index`. A reply is accepted only if the CRC-32 of the raw `data` text as received (never re-serialised) matches, rows have the right width and values are 24-bit. Each accepted reply becomes **one SQLite transaction**: one compact row with the new samples (delta/varint), the loss records for any gap before them, and the sensor's `next_index`. Duplicated or late replies are self-describing (`first_index`), so they can never shift data; a block may only start at or after the committed `next_index` and any overlap is trimmed inside the transaction, so a sample is stored at most once. Stalls hit a 2 s monotonic timeout and are retried; a sensor that stops answering while the other answers is polled less often so it cannot starve the healthy one; after repeated error replies the client re-checks the session. A new `session_id` closes the old session with `[next_index, null]` declared lost. The collector never touches the network.

Loss ledger. For every session and sensor, every index below `next_index` is stored or inside exactly one loss record with a reason (the mock's `lost` reason where it gave one, e.g. `buffer_overflow`, `sensor_outage`; otherwise `not_delivered_by_mock`, `mock_session_restarted_unread_tail` or `disk_full`). Nothing is filled in. `pipeline verify` checks this invariant and SQLite's integrity.

Cloud. Batches merge idempotently into 400-sample segments (presence bitmap + compressed values; a duplicate never overwrites). A window is computed only once all its inputs are *resolved* (held, declared lost, or past the end of a closed session), so every stored result is final. Device and cloud run the same `core` code; heart-rate peaks equal a scan from index 0 (local anchor search, cross-checked against a global scan in tests).

Switching and catch-up. Two failed heartbeats → offline, two good ones → online; in-flight uploads are aborted on every switch, and failed DNS lookups are not cached (the JVM default would hold a failure for 10 s after the network returns). Offline, the device computes windows itself. Back online, the live lane restarts at "now − 10 s" (so live windows have their 5 s heart-rate context) while the catch-up lane asks which ranges the cloud holds and sends the difference in 40,000-index gzip batches, halving the batch on timeout and backing off on errors.

Interpretations (each switchable by one config key, same value on device and cloud): rounding is round-half-even on the exact double (Python `round`); only sensor 1 gaps null the heart rate; result fields are flat (`mean_1`, `min_1`, …); a window with no data on a sensor is reported with `missing_N = 400` and nulls; the last window of a closed session is reported with indices past the end counted as missing; windows restart at 0 in each session.

Measured (stand-ins, local network): online results 0.51 s (median) and at most 1.5 s after a window ends; offline 5.5 s after a network cut, online 1.0 s after it returns; a 60 s outage fully synced 2.0 s later; a 2-hour backlog (5.77 M samples, 28.5 MB on the wire) synced 11 s after restore, and **156 s with the cloud limited to 0.1 CPU / 512 MB** on PostgreSQL; three `kill -9` restarts, every injected mock fault, disk full and ±1 h clock jumps with zero silent loss, zero duplicates and every result equal to an independent reference.

What is guaranteed, and under which assumptions

Claim	Basis	Assumes
A sample acknowledged by a commit survives a crash, reboot or power cut	WAL with `synchronous=FULL` fsyncs every commit	the storage honours flush (some SD cards do not)
Every sample is on disk at most once	blocks start at or after the committed `next_index`; overlap trimmed in the same transaction	the database is not corrupted
No silent loss: every missing index is listed with a reason	ledger invariant, written in the same transaction	as above
A daemon restart loses nothing if it is back within ~8 s	resume from `next_index` via `from_index`; the mock buffers 10 s	the mock kept running (systemd restarts in ~1 s)
Offline within 10 s, online within 10 s of the network changing	≤ 2 s + 2 s + 1.5 s to go offline; ≤ ~4 s to go online	the cloud is awake (a sleeping free-tier service needs ~1 min to wake)
Cloud copy complete after an outage	range reconciliation repeats until nothing is missing	the cloud is reachable and fast enough
Results identical online and offline	same code, computed only on resolved inputs	the interpretations above match the reference

Not guaranteed. (1) Data the mock generated but we had not yet committed when power is lost. (2) The unread tail of an interrupted session is listed only once the device sees the new session. (3) If the disk stays full beyond a 2-minute memory buffer, the oldest buffered samples are dropped and declared `disk_full`; if the process dies while buffering, those samples later appear as the mock's `buffer_overflow`. (4) Online latency and the switch back to online depend on the cloud host; on Render's free tier the service sleeps after 15 min without requests (an uptime pinger prevents it). (5) One cloud instance only. (6) Wall-clock time is display-only; all timeouts and ordering use the monotonic clock.

Maximum data loss on a power cut

Data reaches disk only by commit, and a commit happens every poll (0.5 s when caught up) plus the time to read and commit one reply (normally 5-30 ms). At the moment of a power cut the samples generated since the last commit are either in the mock's memory or in ours, and both are gone (the mock runs on the same board and restarts with a new session). The loss is therefore **one poll interval plus one read-and-commit, about 0.5-0.6 s per sensor**; the largest gap between commits measured in all runs without an injected stall was 0.52 s. We state **at most 1.0 s per sensor**, leaving margin for scheduling or I/O stalls on the Pi. The lost range is recorded at the next boot as `[next_index, null]`, reason `mock_session_restarted_unread_tail`. The simulated power cut (SIGKILL of daemon and mock together) lost at most 0.20 s per sensor in the final run. The bound assumes the storage honours fsync and the disk was not full.

Resource use (online, mock with `--duration 3600`)

Environment (none is a Pi 4)	Avg CPU (of 4 cores)	Peak RSS	Disk written	Network sent
amd64 container, native x86-64 laptop, 4 CPUs / 2 GB, 15 min, local cloud	0.6 % (2.3 % of one core)	109 MB	176 MB/h	15.6 MB/h
ARM64 container under QEMU emulation on that laptop, 4 CPUs / 2 GB, 1 hour	1.6 % (6.3 % of one core)	270 MB	173 MB/h	15.2 MB/h

Environment (none is a Pi 4)	Avg CPU (of 4 cores)	Peak RSS	Disk written	Network sent
GitHub Actions native ARM64 runner, 4 CPUs / 2 GB, 1 hour, live cloud	*pending the workflow run*			

How they relate to a Pi 4 (Cortex-A72, 1.5-1.8 GHz, SD card): the amd64 and native-ARM64 CPU figures **understate** Pi 4 load, because both run on cores several times faster than a Cortex-A72; the emulated CPU and RSS are **inflated** by binary translation relative to native execution on the same laptop, and the Pi's slower cores push the other way, so that row bounds nothing. RSS, bytes written and bytes sent depend on the code and settings, not the core, so the native-ARM64 row (same instruction set and JVM as the Pi) is the best estimate of those; an SD card's internal write amplification is not visible in any row.

Measured with `scripts/measure-resources.sh` (`/proc/PID/stat`, sampled RSS, `/proc/PID/io` `write_bytes`, `/proc/net/dev`; elapsed time from `/proc/uptime`). Writes are SQLite's: one fsynced transaction per reply (~4/s), each rewriting a few 1 KB pages in the WAL and again at checkpoint. Storing one compact row per reply instead of one row per sample cut writes from 336 to 176 MB/h at the same CPU; 1 KB pages had already cut them from 573. The workflow `.github/workflows/arm64-evidence.yml` produces the native-ARM64 row and the live-cloud figures (`evidence/SUMMARY.md`).